How this site was built

A study guide for talking about the portfolio in an interview — the decisions, the mechanisms, and the bugs worth telling someone about.

danieltizon.vercel.app github.com/danil-tzn31/portfolio September 2026

How to use this

You built this site, so none of it should be news. The point of writing it down is that **knowing something and being able to say it under pressure are different skills**, and interviews test the second one.

Work through it like this:

- **Section 1** is a script. Say it out loud until it stops sounding memorised. It is the answer to "tell me about this project", which you will be asked every single time.
- **Sections 4 and 7** are the ones that win interviews — a mechanism you can draw, and a bug you diagnosed. Pick two of each and know them cold.
- **Section 11** is vocabulary. If you use a term, you must be able to define it. If you can't, don't use it.

Everything here is measured or is in the repo. Do not claim a number you have not seen yourself — if someone asks how you know, "I measured it with Lighthouse against a production build" is a good answer and "I read it somewhere" is not.

Contents

1. The sixty-second answer
2. The build at a glance
3. What each section demonstrates
4. Three mechanisms you should be able to draw
5. Performance
6. Accessibility
7. Six bugs worth telling someone about
8. Trade-offs you chose, and how to defend them
9. "Did you use AI to build this?"
10. Rapid-fire questions
11. Vocabulary you must be able to define
12. What you would do next

1 The sixty-second answer

Asked at the start of nearly every technical conversation. Have it ready, keep it short, and stop talking so they can pick a thread.

SAY SOMETHING LIKE THIS

"It's a single-page portfolio built with Next.js, TypeScript and GSAP. The idea was to make it look like a photocopied zine — black and white, halftone images, paper panels — rather than a typical portfolio template.

The part I'd point at is the projects section. It's a photographic contact sheet that scrolls under a fixed magnifying loupe, and the loupe genuinely magnifies — it's a second copy of the strip, scaled, clipped to the lens, and driven by the same tween as the original so the two can't drift apart.

The images are processed at build time into two states: a greyscale plate and a one-bit halftone that resolves into it. That's a script using sharp with an ordered dither, not a CSS filter, because doing it live on a 2000-pixel screenshot costs more than the images do.

It scores 100 on accessibility and 98 on desktop performance. Mobile is lower and I know exactly why — there's a two-and-a-half second entry animation, and that's a deliberate trade I made."

Three things that answer is doing on purpose:

- It names **one** thing to go deep on rather than listing five. Interviewers follow the thread you hand them, so hand them the strongest one.
- It gives a **reason** for a technical choice (build-time, not runtime) instead of just naming a tool.
- It volunteers the **weakness** before they find it. Saying "mobile is lower and here's why" reads as judgment. Being caught by it reads as luck.

2 The build at a glance

Stack, and why each piece is there

Choice	Why
Next.js 16.3.4 App Router	Three things would otherwise be hand-rolled: `next/font/local` for self-hosted subset fonts, `next/image` for the image pipeline, and file-convention metadata (sitemap, robots, social card). Vercel deploy is one command. The site is statically generated — there is no server logic.
TypeScript, strict	No `any`. The project and stack data are typed, so a missing field is a build error rather than a blank space on the page.
Tailwind v4	CSS-first configuration. Design tokens live in one CSS file and are mapped into Tailwind through `@theme`, so a colour or size exists in exactly one place.
GSAP 3.15 + ScrollTrigger, SplitText	Pinning, scrubbing and text splitting in one library with one timing vocabulary. Only two plugins are registered — two more were registered early on and called nowhere, and removing them was worth about three points of mobile performance.
Lenis 1.3.26	Smooth scroll, driven off the GSAP ticker rather than its own loop so the two never fight.
sharp (dev only)	Build-time image processing. Not shipped to the browser.

Six runtime dependencies in total — Next, React, React DOM, GSAP, the GSAP React hook, and Lenis. No component library, no icon package, no utility library, no state manager.

> If asked "why not Framer Motion as well?": mixing two animation libraries in a site this size means two timing systems, two easing vocabularies and two ways of handling reduced motion. Both are good libraries. Using both here would have been a mess, and it is the loudest "assembled from tutorials" signal on a junior portfolio.

Size of the thing

Measure	Value
Source files	46 (`.ts` , `.tsx` , `.css`)
Lines of source	~3,100 non-blank, of which ~15% are comments
Routes	One page, plus generated icon, social card, robots and sitemap
Rendering	Fully static (SSG) — no server-side logic at request time

What the browser downloads on first load

Asset type	Transferred
JavaScript	211 kB
Fonts (5 subset woff2)	107 kB
CSS	43 kB
HTML	27 kB
Images above the fold	25 kB
Total	**~421 kB**

Project media loads as you scroll: greyscale plates are 32–114 kB AVIF, halftones 9–35 kB PNG, contact-sheet thumbnails 2–5 kB.

Lighthouse, against a production build

	Desktop	Mobile
Performance	98	77–81
Accessibility	100	100
Best practices	100	100
SEO	100	100
Cumulative Layout Shift	0.001	0.001

> These were taken locally against `next start`, not `next dev`. Development builds are not optimised and scoring one is meaningless. If you re-run against the live URL the numbers will differ slightly — quote whichever you actually ran, and say which.

3 What each section demonstrates

If someone scrolls the site while you talk, this is what to point at.

Section	The technique
Preloader	The hero's own `<h1>` is what assembles — there is no separate loading screen. SplitText, a hand-authored table of entry offsets, and a colour boundary that tracks a rising curtain.
Hero	Fluid type sized from two competing constraints at once. Two-layer parallax on scroll-out. A vertical role slip.
About	Line-masked text reveals using SplitText's own `mask` and `autoSplit`, so a resize or a late font load re-splits cleanly instead of leaving doubled text.
Tech stack	Three marquee rows whose speed is driven by scroll velocity — scroll fast and they surge and briefly reverse, then ease back to their idle speed.
Projects	The centrepiece. A pinned, scrubbed contact sheet under a real magnifier, with hard scan-bar cuts between projects instead of crossfades.
Contact	Copy-to-clipboard with a stamped confirmation rather than a toast, and an `aria-live` announcement for screen readers.

Two systems that run through all of it

Motion tokens. Every duration and easing curve on the site lives in one file, `lib/motion.ts`, as named constants — `D.base`, `E.settle`, `SCRUB.tight`. No timing number appears anywhere else in the codebase. That is what makes the motion feel like one system rather than a series of unrelated animations, and it means retuning the feel of the whole site is one file.

Design tokens. Five colours and no sixth. One misregistration offset (`--slip`) and one paste angle (`--tilt`), reused everywhere — a single repeated value reads as a printing fault, while varied random values just read as noise.

4 Three mechanisms you should be able to draw

Being able to sketch one of these on a whiteboard is worth more than naming ten libraries.

4.1 The loupe — the strongest thing to talk about

The problem. A contact sheet of project thumbnails scrolls vertically under a fixed magnifying glass. Inside the glass, the strip has to appear larger — and the frame under the glass must always be the same frame that is passing behind it, at every scroll position.

The structure. Two copies of the same strip:

```
.sheet                      overflow: hidden
  .sheet__strip             the real strip, translates on scrub
  .sheet__lens              absolutely positioned, overflow: hidden
    .sheet__strip--ghost    a duplicate, scale(1.35),
                            transform-origin fixed at the lens centre
```

The bug that makes this a good story. The obvious implementation writes the same `y` to both strips. That puts them in register at exactly one scroll position and nowhere else. The reason is that in a CSS transform, **the translation is not scaled by the scale beside it** — they are separate components of the matrix. So the copy has to travel the original distance *times the magnification*:

```
onUpdate: () => {
  gsap.set(strip, { y: travel.y });
  gsap.set(ghost, { y: travel.y * MAG });   // not travel.y
}
```

Both are written by a single tween, so they cannot drift apart. Measured registration error after the fix: **0.000 px at every scrub speed**, including a hard reverse scrub.

The invariant to state if pushed. It is not "the two strips agree". It is: *the copy's offset equals the frame's distance from the lens centre, multiplied by (MAG − 1)*. That is also why the transform origin has to be pinned to the centre of the lens — any other origin makes the two agree in one place only.

Two details worth mentioning. The lens is measured from the image plate, not from the whole frame including its caption — otherwise the magnified area is not square with what it is magnifying. And the thumbnails are screened at their own, coarser resolution, because a 1600-pixel dither displayed at 280 pixels is texture, not dots, and there is nothing for a magnifier to enlarge.

4.2 The preloader curtain

The problem. A black curtain covers the hero. The wordmark sits above it and must be *paper-coloured where the curtain still covers* and *ink-coloured where the page is already showing* — and during the rise it is both at once, split across a moving line.

What does not work. `mix-blend-mode: difference` produces the inversion automatically, and it was tried. In motion, a letter split across a moving edge reads as a rendering fault rather than a reveal.

What does. A two-stop hard-edge gradient clipped to the glyphs:

```
.wordmark--curtain {
  color: transparent;
  background-image: linear-gradient(
    to bottom,
    var(--paper) 0 var(--curtain, 100%),
    var(--ink)   var(--curtain, 100%) 100%
  );
  background-clip: text;
}
```

`--curtain` is the curtain's bottom edge in the heading's own coordinates, written every frame by **the same tween that moves the curtain** — so the two cannot drift.

Three things that silently break it, all worth knowing because they are the kind of detail an interviewer probes for:

- SplitText must be reverted *before* the curtain rises. `background-clip: text` does not follow transformed inline-block children, and every character is one until the split comes out.
- The inline `color` set during the entry has to be cleared, or it outranks the class and paints the glyphs opaquely over the gradient.
- The page must be scrolled to the top, with the browser's scroll restoration turned off, *before* the scroll lock is applied — otherwise a refresh below the fold leaves the visitor frozen mid-page while the animation plays off-screen above them.

The safety net. Nothing on this page is worth a permanently black screen, so: the font gate waits on the wordmark's own face only, raced against a 900 ms timeout; the font promise is `.catch()`-ed, because an uncaught rejection would take the chain with it; and a five-second failsafe reveals the page whatever else has gone wrong.

4.3 The halftone pipeline

Where it runs. A Node script using sharp, run by hand; the output is committed. It is not part of the build. Doing this live as a CSS or SVG filter on a 2000-pixel screenshot costs more than the images themselves.

Three findings, each of which decides whether it reads as print or as mud:

- **Ordered dither, not error diffusion.** An 8×8 Bayer matrix gives the regular crosshatch a photocopier makes. Floyd–Steinberg gives a photographic scatter that reads as noise.
- **The threshold cell must be wider than one pixel.** At one pixel it looks perfect at 1:1 and turns to grey mush with moiré banding the moment the browser downscales it — which it always does. Two pixels survives the downscale.
- **Each image is exposed individually,** binary-searching its own brightness lift to land the same proportion of the frame in ink. One fixed curve cannot serve a bright photograph and a near-black interface, and the contact sheet shows all four at once, so the difference reads as three of them being wrong.

And a format decision worth defending: the halftones are PNG, not AVIF. A one-bit image is nothing but hard edges, so a lossy codec both blurs the screen and — here — encodes about twice as large. The

greyscale plates are AVIF, where lossy compression is exactly right.

5 Performance

What the numbers are, and what causes them

Desktop performance is 98. Mobile is 77–81, and the cause is a single thing you chose deliberately: **the 2.5-second entry animation**. Largest Contentful Paint on mobile is around 2.8 seconds, of which nearly all is element render delay against a time-to-first-byte of about 22 milliseconds. Nothing on the network and nothing in the bundle is responsible.

> IF ASKED WHY MOBILE IS LOWER
>
> "It's the entry animation. LCP is about 2.8 seconds and almost all of it is render delay — TTFB is 22 milliseconds, so it isn't network. Shortening the entry to around a second would put mobile above 90. I kept it because the animation is the first impression, but it's a trade, not an accident, and I can show you the waterfall."

That answer is strong because it demonstrates you know the difference between *a slow site* and *a site that chooses to wait*. Most people cannot tell those apart from a score.

Decisions that cost nothing and bought a lot

- **Cumulative Layout Shift of 0.001.** Every image has explicit dimensions, and the fonts are self-hosted and preloaded so there is no reflow when they arrive.
- **The SVG ink-bleed filter is gated behind a class** added only after the preloader finishes. SVG filters on text are a known performance trap; this way the filter can never delay the first paint of the largest element on the page.
- **Fonts are subset** — five faces, roughly 397 glyphs each, 107 kB in total for a family pairing that would otherwise be several hundred.
- **Two unused GSAP plugins were removed** once it was clear nothing called them. They were being parsed and evaluated on every load for nothing. Worth about three points on mobile.
- **The grain overlay animates `background-position` through four fixed offsets with a CSS `steps()` animation** — no JavaScript, no canvas, and it repaints only a composited layer.

The honest weakness

JavaScript is 211 kB transferred, against a target of 200. It is not egregious, and most of it is React plus GSAP, but if asked "what would you cut", that is the honest place to start — and the honest answer is that GSAP's ScrollTrigger is doing enough work here to justify its size, while the entry animation is the thing actually costing the score.

6 Accessibility

100 on a page with a pinned scrub section, which is the hardest kind of page to get there. This is worth raising unprompted — very few portfolios do it, and it is a strong signal about how you work.

What was done

- **Every contact-sheet frame is a real link** and all four are focusable at all times, in reading order, on every breakpoint. The big detail panel is a visual enhancement on top of that, not the only way in.
- **Focusing a frame moves the timeline to that project**, so tabbing visibly walks the loupe down the sheet. The keyboard sees what the mouse sees rather than a static frame with four invisible links on it.
- **The duplicated ghost strip is `aria-hidden` with its links removed from the tab order,** so a screen reader hears four projects, not eight. The same applies to the marquee: the visible track is decorative and duplicated, and a visually-hidden list carries the real content.
- **Reduced motion is handled everywhere**, and in the projects section it is handled structurally — below the breakpoint and under reduced motion, the whole section is plain CSS with no pin, no scrub and no entry animations.
- **Every colour pair was measured,** not assumed. The lowest ratio on the site is about 5.6:1, and most type is far above that.

Two traps worth being able to describe

SplitText rewrites the accessible name. By default it writes an `aria-label` onto the element it splits. On a `<p>` that attribute is prohibited outright; on a button it replaced the name with text that did not contain the visible words, which fails the "label in name" criterion. Setting it to announce nothing removed a control from the accessibility tree entirely. The fix is structural: **split a span inside the control, never the control itself**, with the span already hidden from assistive technology. The rule to take away is to check the accessible name *after* splitting text, not before.

`autoAlpha` makes things unfocusable. GSAP's `autoAlpha` sets `visibility: hidden` at zero opacity, and a hidden control cannot receive focus. An entry animation on the contact rows meant a keyboard user could not reach the email button or either link until something scrolled the section into view for them. The animation was removed — the addresses inside already arrive on their own.

7 Six bugs worth telling someone about

"Tell me about a bug you fixed" is asked constantly, and most candidates answer badly — they describe the symptom and the fix, and skip the diagnosis. The diagnosis is the entire point of the question. Learn two of these properly.

1 The pinned panel overflowed, and every measurement said it fitted

SYMPTOM — Content spilled off the bottom of the pinned projects sheet, but measuring the panel against its container said it fitted exactly.

DIAGNOSIS — The panel sat in a CSS grid, and a grid's implicit row is `auto` — sized by its tallest item. So the row grew to fit the content, the panel grew with it, and the panel measured as fitting a container that was itself hanging off the sheet.

FIX — `grid-template-rows: minmax(0, 1fr)`, plus `min-height: 0` on every flex and grid item down the chain.

LESSON — A child that fits its parent proves nothing when the parent is what overflows. Measure the deepest content against the thing that must actually contain it.

2 A one-pixel white line under the torn paper edge

SYMPTOM — A hairline of light colour ran the full width just above the torn edge between two sections.

DIAGNOSIS — The first assumption was antialiasing — a similar hairline earlier in the build *was* antialiasing, caused by a mask's straight edge landing exactly on the element boundary at fractional device pixel ratios. Sampling the tear at four different pixel ratios found nothing. The line was a *paper-coloured layer sitting behind the section*, left over from an earlier design where the contact section arrived as an inversion.

FIX — Removing the inversion removed the layer, and the line with it.

LESSON — When a one-pixel line appears, check what is *behind* the element before adjusting the element. The previous fix being right once made it the wrong first guess the second time.

3 A slow animation that felt fast

SYMPTOM — The curtain rise felt like a quick whip with dead air around it, and doubling its duration did not help.

DIAGNOSIS — Rather than guessing, the panel's position was sampled frame by frame. With `power3.inOut` it sat nearly still for the first 260 ms, covered 60% of the distance in the middle 200 ms, then crawled the last 6% over 300 ms. The problem was the shape of the curve, not the length of it.

FIX — `power1.inOut`, whose peak velocity is about 1.5× its average rather than 4×. The same sampling then gave an even progression.

LESSON — If a move feels faster than its duration says, sample its position rather than adding time to it.

4 An asterisk turned into an emoji, but only on iPad

SYMPTOM — Two separator characters rendered as flat black marks on Windows and as colour emoji on iPadOS.

DIAGNOSIS — Both characters are classified in Unicode as emoji-capable with default *text* presentation, meaning a browser may legitimately render either form. Neither is present in the subset webfonts — confirmed by reading the fonts' character maps directly — so the browser fell through to a system font. On Windows that resolved to a symbol font; on Apple platforms the only font covering those codepoints is Apple Color Emoji.

FIX — Drew the asterisk as an inline SVG path. A path cannot be substituted by a font. The variation selector that requests text presentation was rejected as a fix because it only *asks*, and cannot work when no text font in the chain covers the glyph either.

LESSON — A subset font means anything past basic Latin escapes into a system font, and which system font is the device's choice, not yours.

5 A null-coalescing fallback that always ran both branches

SYMPTOM — Caught by the linter as an unused expression before it could cause a visible bug.

DIAGNOSIS — The code was `getLenis()?.scrollTo(to) ?? window.scrollTo({ top: to })`. `scrollTo` returns `undefined`, so `??` never short-circuits and both scrolls always ran — a scroll fighting itself.

FIX — A plain if/else.

LESSON — `??` falls back on `null` and `undefined`, which includes the return value of every void function. It is a fallback for *values*, not for *actions*.

6 Measuring the wrong server twice

SYMPTOM — Lighthouse kept reporting accessibility failures whose text had already been fixed.

DIAGNOSIS — A stale production server was still holding the port, so the rebuilt output was never the thing being served.

FIX — Check the build ID and grep the served HTML for a string you just changed before trusting any number from it.

LESSON — Worth telling because it is about measurement discipline rather than code. Before believing a bad result, confirm you measured the thing you think you measured.

8 Trade-offs you chose, and how to defend them

Every one of these is a place an interviewer can push. A trade-off you can name and justify is a strength. The same trade-off you cannot explain is a weakness.

The 2.5-second preloader

Cost: roughly twenty points of mobile performance score. **Defence:** it is the site's identity and it plays on every load by design. **Be honest that:** if this were a client's e-commerce site it would be indefensible, and you would cut it immediately. Knowing when a decision is context-dependent is the actual answer here.

Next.js for a single static page

Defence: font subsetting and loading, the image pipeline, and metadata file conventions would all be hand-rolled otherwise, and the deploy is one command. **Be honest that:** the site does not need a framework — a hand-written page would work. The framework bought tooling, not capability.

No tests

Defence: it is a static one-page site with no business logic and no user input beyond a copy button. A test suite on this would be theatre. What *was* verified, and repeatedly, was measured: element geometry, transform matrices, drift rates, contrast ratios, overflow at seven viewport sizes, accessibility tree contents. **Be honest that:** on a codebase with logic, you would expect to write tests, and you should say so plainly rather than defending their absence in general.

An unconventional structure

Defence: the brief was explicitly to not look like a template, and the navigation is a table of contents in the right margin — a four-hundred-year-old convention, which is exactly why it does not read as a nav bar. **Be honest that:** unconventional layouts cost discoverability, which is why every section still has a real heading, a real link, and a keyboard path.

9 "Did you use AI to build this?"

In 2026 this is a routine question for any portfolio with this level of polish, and the wrong answer is a defensive one. Answer it the same way you would answer "did you use Stack Overflow" — matter-of-factly.

A GOOD SHAPE FOR THE ANSWER

"Yes, I used AI as a tool throughout, the same way I use documentation and a linter. What it didn't do was make the decisions. I set the art direction, I rejected approaches that didn't work, and I'm the one who caught the things that were wrong.

For example, the colour boundary in the preloader was originally done with `mix-blend-mode: difference`. It worked, technically. It looked like a rendering fault in motion, so I threw it out and rebuilt it as a gradient clipped to the text. That call — and knowing why the first one felt wrong — is the part that's mine.

Ask me anything in the repo and I can tell you why it's there."

Why that answer works: it is honest, it is specific, and it ends with an invitation. Vagueness is what reads badly here, not the tool.

What makes it true — and what you need to be able to back up. The strongest evidence is that you can explain a decision that was *reversed*, and why. Have two or three ready:

- The blend-mode inversion, thrown out because it read as a glitch in motion.
- The mobile projects layout, where the animations and the small-screen index were cut entirely because they were fighting the page rather than helping it.
- The contact section, rebuilt from an inverting panel to a single flat ink page — which also happened to remove a one-pixel line that had been chased as an antialiasing bug.

> The real preparation here is not rehearsing an answer. It is being able to open any file in the repo and say what it does and why. If there is a file you cannot explain, read it before the interview — that is the actual gap, and it is fixable in an afternoon.

10 Rapid-fire questions

Short answers. If one of these makes you hesitate, that is the thing to go and read.

Why is the site statically generated rather than server-rendered?

There is no per-request data. Everything is known at build time, so it is prerendered once and served from a CDN.

What is ScrollTrigger actually doing in the projects section?

Pinning the section for four screens of scroll, and scrubbing a timeline against that scroll rather than playing it on a clock. Snap settles it to the nearest project when the wheel stops.

What does "scrub" mean?

The animation's playhead is tied to the scroll position instead of to time, with a short catch-up so it eases rather than tracking rigidly.

Why drive Lenis from the GSAP ticker?

So there is one animation frame loop rather than two competing ones. GSAP's lag smoothing is turned off because it would fight Lenis's own interpolation.

How do the images get to be one-bit?

A build-time script thresholds each pixel against an 8×8 Bayer matrix, at two pixels per threshold cell, after an exposure adjustment chosen per image.

Why AVIF for one image and PNG for the other?

The plate is a photograph, where lossy compression is efficient. The halftone is pure hard edges, where a lossy codec blurs the screen and encodes larger.

How are the torn paper edges made?

Four hand-drawn SVG paths, each a 600-pixel tile, applied as a repeating `mask-image` over a solid colour — repeated rather than stretched, so a tear looks identical on a full-width panel and a narrow one.

What happens on a phone?

The pin, the loupe and the section's animations are not created at all — the layout below the breakpoint is plain CSS. It is a structural split via `gsap.matchMedia()`, not the same layout with different numbers.

What happens with reduced motion?

The same branch. Reduced motion is treated as another breakpoint rather than as an afterthought, which is why it needed no special handling in the page index at all — nothing there tweens.

Why self-host the fonts?

They are licensed files, not a web service, and self-hosting means no third-party request on the critical path. They are subset to the characters actually used.

What is the largest contentful paint element?

The hero heading — real text, painted in the first frame. The black curtain is deliberately a sibling behind it in stacking order, because if the curtain were the LCP element the score would collapse and the page would read as a loading screen.

What would you do differently?

Shorten the entry animation, and build something with real users next rather than another self-directed project. (Say this one plainly. It is the single best answer in the whole document.)

11 Vocabulary you must be able to define

The rule: if you say it, you can define it. If you cannot define it, do not say it — reach for the plain-English version instead, which is almost always the better answer anyway.

Term	One-sentence definition
SSG / prerendering	The page's HTML is generated once at build time and served as a file, rather than assembled per request.
Hydration	The browser attaches JavaScript behaviour to HTML that was already rendered, so the page is visible before it is interactive.
LCP	Largest Contentful Paint — when the biggest visible element finishes rendering. A proxy for "when did this look loaded".
CLS	Cumulative Layout Shift — how much content jumps around after it appears. Ours is 0.001; anything under 0.1 is good.
TTFB	Time to First Byte — how long the server took to start responding. Ours is about 22 ms, which is why the slow LCP is clearly not a network problem.
Pinning	Fixing a section in place while the page scrolls past, so scroll distance becomes animation progress.
Scrubbing	Tying an animation's playhead to scroll position instead of to elapsed time.
Ordered dither	Turning greyscale into pure black and white by comparing each pixel against a fixed repeating threshold pattern, producing a regular crosshatch.
Subset font	A font file stripped to only the characters a site actually uses, to cut its size.
`mask-image`	A CSS property where one image's alpha decides which parts of an element are painted — used here for the torn edges.
`background-clip: text`	Painting an element's background only inside the shapes of its glyphs, so text can carry a gradient.
Stacking context	A self-contained layer group; `z-index` only compares elements inside the same one, which is why the curtain and the wordmark have to share a parent.
Composited layer	An element the browser can move or fade on the GPU without re-running layout or paint — which is why transforms and opacity are cheap to animate and other properties are not.
Accessible name	The label assistive technology announces for a control, computed from its content and attributes — and overwritten by `aria-label`, which is the trap described in section 6.
WCAG contrast ratio	A measure of how distinguishable two colours are, from 1:1 to 21:1. Body text should be at least 4.5:1.

12 What you would do next

Expect "what would you improve?" — and note that a candidate with no answer sounds either incurious or defensive. These are real, in order.

1. **Shorten the entry animation, or make it shorter on repeat visits.** It is the single change that moves the mobile score most, and it is a choice rather than a defect.
2. **Build something with users who are not you.** An accepted open-source pull request, a small paid client, anything with a second stakeholder. This is the biggest genuine gap in the whole portfolio, and it is worth saying so out loud before anyone else does.
3. **Trim the JavaScript.** 211 kB against a 200 kB target. Not urgent, but it is the honest answer to "what's over budget".
4. **Write about one of these builds.** The site's own statement is "learning out loud, in public, on purpose", and there is currently nothing public to read. One good post about the loupe would close that gap and would double as interview preparation.
5. **Add a downloadable CV to the contact section** once it is revised, so the site works in both directions.

One last thing

The most common way a good portfolio fails in an interview is that the candidate describes *what* they built and never *why*. Every section of this document is arranged around a reason, because the reason is the part that is actually yours — anyone can install GSAP, and almost nobody can explain why the copy of the strip translates at a different rate from the original.

Know two mechanisms and two bugs cold. Volunteer one weakness before you are asked. Then stop talking and let them ask.